

Solutions to the challenges proposed by 0xPARC

Exact representations, index deconvolution and constrained computation

Alberto Hernandez-Espinosa

17 September 2026

Abstract

In this report I present possible solutions to five of the eight challenges raised by 0xPARC [1]: the majority construction and the four integer conditions expressed in a finite field. These are the questions one method answers. I use an index-deconvolution method to recover a Boolean system's functional description from its behaviour alone; run on a canonical decision diagram it does so without enumerating states. The same method supplies the gates for the arithmetic questions: each cell is stated as an integer relation and whatever gate the method returns is what is compiled, so the classical full adder is recovered rather than assumed. The remaining three questions have answers that this method does not supply, and reporting them here would credit it with work it did not do. Supplementary Information gives the proofs, complete specifications and verification evidence.

1 Disclaimer on the use of AI

AI tools were used as assistants for constructing the testbed, searching for and corroborating bibliography and related work, discussing the solutions presented here, and reviewing grammar and phrasing. The ideas, research, methods, validation, interpretations and claims are original to the author, who takes responsibility for the final content and for independently checking the cited sources.

2 What index deconvolution is

Index deconvolution is the Boolean-network component used throughout this response. Its lineage is algorithmic information dynamics: the study of how an integrated system responds to interventions, how a generative description exposes behaviour, and how such descriptions support explanation and prediction [2, 3]. It takes information to be program size, in the sense of Kolmogorov and Chaitin [5, 6, 7], rather than a distribution over states, and it specialises that view to exact finite Boolean descriptions; it does not compute Kolmogorov complexity itself. In practice it compresses the behaviour of a complex Boolean network into a construction small enough to answer the queries that matter, yet faithful enough to express the behaviour of the system exactly. Single-bit perturbations isolate the responsive coordinates, the restricted behaviour is matched against a canonical gate family, and the recovered description is replayed forward, where it must reproduce the original behaviour exactly.

The method is under active development, and a full account of it is in preparation for publication; what is used here is the part that is settled, and the implementation accompanies this response [10].

This is what decides which questions appear below. Four of them are answered by an object the method returned, and the fifth, Q8, is not answered by it at all: there the contribution is a measurement of why, and the answer is the direct limb construction. Questions whose answers

the method does not supply are left to another account rather than reported here with the method attached to them. Supplementary Sections S7–S9 record, answer by answer, what the method supplies, together with the evidence, the denominators and the limits. Section S9 in particular separates what the strongest of those measurements does and does not establish.

3 Constructing and recovering majority (Q2)

A three-input majority gate returns one when at least two inputs are one. *The question asks whether these gates can compute majority on 2025 inputs, with free duplication of wires.* **They can.** The useful identity can be understood by temporarily allowing a more general monotone Boolean function f : changing an input from zero to one never lowers its output.

Choose three distinct inputs a, b, c . Form three simpler functions by replacing b with a , c with b , and a with c , respectively. Then

$$f = \text{MAJ}_3(f_{b \leftarrow a}, f_{c \leftarrow b}, f_{a \leftarrow c}). \quad (1)$$

If the three input bits agree, all substitutions preserve the assignment. Otherwise, one preserves it, one increases a bit and one decreases a bit. Their outputs therefore bracket the original output, with the original itself among them. Their majority equals f .

Each substitution removes an active input. Odd majority is also self-dual: complementing every input complements the output. Both properties survive substitution, and a monotone self-dual function on at most two inputs is just an input wire. Repeating Eq. (1) therefore produces a circuit containing only three-input majority gates. The proof applies to every odd input count, including 2025. It gives existence; its loose size bound does not establish an economical circuit at that scale.

3.1 A network we can inspect completely

For five inputs, the construction emits the seven gates in Figure 1. Each arrow carries an input bit or the output of an earlier gate. For example, $g_0 = \text{MAJ}_3(x_0, x_2, x_4)$ and $g_1 = \text{MAJ}_3(x_0, g_0, x_3)$. The final gate takes g_1, g_3, g_5 . Reading across a row of the connection matrix identifies the three sources of that gate; evaluating the gates in order gives the output.

Take input 11100, written in the order $x_0, \dots, x_4$. Its first three bits are one, and its output is one. Now vary the last two bits. The four inputs 11100, 11110, 11101, 11111 still have output one, although some internal gates change. This is the pattern 111*: either value is allowed at each star, which is a don't-care position in the notation of Holland's schemata [4]. The network shows how the answer is produced, while the pattern identifies which variations preserve that answer.

3.2 What index deconvolution adds

A circuit describes how to compute an output. Its repertoire is the complete table of outputs for all input states. Index deconvolution starts from that table and asks which inputs actually affect the result. For a Boolean function f on n bits, encode a state by

$$\iota(x) = \sum_{i=0}^{n-1} 2^i x_i, \quad \mathcal{I}_f = \{\iota(x) : f(x) = 1\}. \quad (2)$$

The index set $\mathcal{I}_f$ records exactly where the output is one; all other indices have output zero. Input i matters if there is a pair of states differing only at i for which the outputs differ. These pairs isolate the dependence of this output condition inside the complete repertoire. They do not

A. Follow the actual majority circuit



Every gate returns the majority of its three incoming wires.

B. Read one target row

Target gate	x_0	x_1	x_2	x_3	x_4	g_0	g_1	g_2	g_3	g_4	g_5
g_0	1	0	1	0	1	0	0	0	0	0	0
g_1	1	0	0	1	0	1	0	0	0	0	0
g_2	0	1	0	1	1	0	0	0	0	0	0
g_3	1	1	0	0	0	0	0	1	0	0	0
g_4	0	0	1	1	1	0	0	0	0	0	0
g_5	0	1	1	0	0	0	0	0	0	1	0
g_6	0	0	0	0	0	0	1	0	1	0	1
	x_0	x_1	x_2	x_3	x_4	g_0	g_1	g_2	g_3	g_4	g_5

Source input or gate

C. The same input pattern can contain different internal gate states

Index	Input $x_0 \dots x_4$	g_0	g_1	g_2	g_3	g_4	g_5	g_6	Output
7	11100	1	1	0	1	0	1	1	1
15	11110	1	1	1	1	1	1	1	1
23	11101	1	1	1	1	1	1	1	1
31	11111	1	1	1	1	1	1	1	1

Figure 1: How does a network of small majority gates compute a larger vote? Panel A shows the actual five-input, seven-gate construction, with arrows from sources to targets. Every gate computes three-input majority. Panel B records the same connections as a matrix; each target row has three ones. Panel C evaluates all four fillings of 111 **. Some internal outputs change while the final output remains one. Inputs read $x_0 \dots x_4$; their index weights are 1, 2, 4, 8, 16.

imply that the integrated network or its evolving behaviour has been reduced to a smaller table: changing the selected node union or dynamical condition changes the observation being asked of the same system. Equality with the original repertoire checks that the stated finite recovery is exact.

A further compression describes groups of output-one states by patterns, or *schemata*. A schema in Holland’s sense [4] is a template over $\{0, 1, *\}$ in which $*$ marks a position whose value does not affect membership, and the correspondence here is exact rather than an analogy. Holland’s $*$ is defined per template, which is why a free position below must likewise be read per schema. Figure 2 follows the same five-input network through its ten schemata and all 32 input states. Each schema fixes three bits to one and leaves two free. Any state with at least three ones belongs to one or more of these patterns; no state with fewer ones belongs to any of them.

For $111**$, the fixed ones give decimal anchor $1 + 2 + 4 = 7$. The *sumandos* are the fillings of the schema’s don’t-care positions. Here their numerical offsets are 0, 8, 16, 24: set neither free bit, only bit 3, only bit 4, or both. Adding these offsets to 7 gives indices 7, 15, 23, 31, the four highlighted rows of the repertoire. The ten schemata together expand to exactly 16 output-one indices. The union counts each index once, even when several schemata contain it.

This anchor-and-offset reading of a schema is the representation I introduced for estimating integrated information from algorithmic complexity and dynamic querying [2, 3], where it extends Holland’s template from a device for classifying a search space into a lossless description of a network’s output repertoire: each schema is carried as its decimal anchor together with its own family of offsets, so that queries about the system are answered from the compressed description rather than from the enumerated repertoire. The present response uses the same representation.

Free positions belong to a particular schema. In $111**$, inputs 3 and 4 can vary without changing the final answer. Both inputs still matter elsewhere in the repertoire. For example, fixing two of the other inputs to one makes flipping either one decisive. Likewise, changing 01100 to 11100 flips only input 0 and changes the output from zero to one. These paired states are what index deconvolution uses to recover functional dependence. The patterns and the perturbation pairs describe two complementary aspects of the same exact map.

For odd majority on n inputs, every input matters: fix the other $n - 1$ bits with exactly $(n - 1)/2$ ones, then flip the remaining bit. Complementation pairs each output-one state with an output-zero state, giving 2^{n-1} active indices. This explains why index deconvolution does not remove inputs from true majority, even though its output set admits schemata with free positions.

Recovery is not limited to circuits small enough to tabulate. Run on the decision diagram, deconvolution recovers the emitted circuit at 151 inputs: every one of the 151 inputs comes back essential, and the recovered object is identical to a threshold function specified by counting rather than by majority gates. The two constructions share no code: one merges weights recursively and emits three-input gates, the other is a dynamic program over the coordinates. That identity settles agreement on all 2^{151} input states at once, and not one of them is enumerated. Naming the recovered function is a weaker act than recovering it, and Supplementary Section S9 says exactly how much weaker.

Table 1 lists the ladder, and it is worth separating its two node columns. Recovering majority from a circuit of 8,116,950 gates allocates 41,196,123 decision nodes, because composing the circuit gives every intermediate wire a diagram of its own; that allocation grows as $n^{3.96}$. The recovered function’s own description is 5,776 nodes and grows as $n^{1.97}$, in fact exactly $(n + 1)^2/4$ at every size measured. The second number is the one the method has to carry, and the reason it stays small is a counting argument: read in order, the only thing a majority must remember is how many inputs have been one so far, so a node is a position paired with a running count. Not every function is so obliging, and Section 4.5 reports one that is not. Supplementary Section S9 separates the four quantities involved, gives the per-size measurements, and states what the

	x_0	x_1	x_2	x_3	x_4
111**	1	1	1	*	*
11*1*	1	1	*	1	*
11**1	1	1	*	*	1
1*11*	1	*	1	1	*
1*1*1	1	*	1	*	1
1**11	1	*	*	1	1
111	*	1	1	1	*
*11*1	*	1	1	*	1
*1*11	*	1	*	1	1
**111	*	*	1	1	1

0	0	0	0	0	0	0
1	1	0	0	0	0	0
2	0	1	0	0	0	0
3	1	1	0	0	0	0
4	0	0	1	0	0	0
5	1	0	1	0	0	0
6	0	1	1	0	0	0
7	1	1	1	0	0	1
8	0	0	0	1	0	0
9	1	0	0	1	0	0
10	0	1	0	1	0	0
11	1	1	0	1	0	1
12	0	0	1	1	0	0
13	1	0	1	1	0	1
14	0	1	1	1	0	1
15	1	1	1	1	0	1
16	0	0	0	0	1	0
17	1	0	0	0	1	0
18	0	1	0	0	1	0
19	1	1	0	0	1	1
20	0	0	1	0	1	0
21	1	0	1	0	1	1
22	0	1	1	0	1	1
23	1	1	1	0	1	1
24	0	0	0	1	1	0
25	1	0	0	1	1	1
26	0	1	0	1	1	1
27	1	1	0	1	1	1
28	0	0	1	1	1	1
29	1	0	1	1	1	1
30	0	1	1	1	1	1
31	1	1	1	1	1	1
	x_0	x_1	x_2	x_3	x_4	y

Pattern 111** fixes three ones.
Anchor: 1 + 2 + 4 = 7
Free positions: 3 and 4
Offsets: {0, 8, 16, 24}
Indices: {7, 15, 23, 31}
The four orange rows all have $y = 1$.

Flip only input 0 at the boundary:
01100 → 11100
index 6, $y = 0$ → index 7, $y = 1$
Input 0 matters to the function.

5

comparison with the threshold specification does and does not establish.

Table 1: Recovering majority from its own behaviour. Each row is the emitted circuit at that size, deconvolved and compared with a threshold specification. Every input is recovered as essential and every comparison is an identity, so the agreement holds on all 2^n states without enumerating any. *Allocated* counts every node the recovery used, most of it the diagrams of the circuit’s own intermediate wires; *description* is the recovered function’s diagram alone. Supplementary Section S9 separates them.

n	Gates	Allocated	Description	States covered
11	205	1,473	36	2,048
31	14,190	77,573	256	2^{31}
63	245,086	1,273,757	1,024	2^{63}
101	1,623,175	8,304,498	2,601	2^{101}
151	8,116,950	41,196,123	5,776	2^{151}

What is recovered is the computed function, not a unique arrangement of internal gates. Several different circuits can share one repertoire.

Nor are an input count and an output frequency sufficient certificates. A majority of three disjoint three-input majorities uses all nine inputs and returns one on half its states, just like true nine-input majority. Yet groups 110, 110,000 give local votes 1, 1, 0 and an incorrect final one: only four original bits are one. Comparing the actual index sets exposes this failure. Supplementary Section S3 gives the reconstruction proof, controls and limits of identifiability.

4 Expressing integer conditions by quadratic equations (Q5–Q8)

These questions use a different, 254-bit prime P , specified in Supplementary Section S1. Arithmetic takes place modulo P . A constraint has the form $A(w)B(w) = C(w)$, where each expression is linear in the variables w . A *witness* is an assignment of all private and auxiliary values satisfying the constraints.

Four integer conditions are asked for: that a number lies below 2^{64} , that a field element is not one, that a 64-bit number has a nontrivial factorisation, and that a 4096-bit number does. I answered them by running a single experiment and then reading its consequences, so I describe it in that order.

4.1 An experiment in which no gate is written down

I set the rule that nothing about *how* to compute an arithmetic cell would appear anywhere in the source. Stated precisely, every cell is supplied *extensionally*. A cell with n input bits and m outputs is a map

$$f : \{0, 1\}^n \rightarrow \{0, 1\}^m, \quad (3)$$

and what the method receives is its graph $\{(x, f(x)) : x \in \{0, 1\}^n\}$, the function as a set of pairs. Writing a gate would instead supply an *intensional* description, a term whose value happens to be f . The experiment withholds exactly that and asks the method to return one. Below, $\bar{x}$ is $1 - x$ and $[\cdot]$ is one when the enclosed statement holds and zero otherwise.

Each f is written as the integer relation it must satisfy. For the adder, on $(a, b, c_{\text{in}}) \in \{0, 1\}^3$, the two outputs are the base-two digits of an integer sum,

$$s = (a + b + c_{\text{in}}) \bmod 2, \quad k = \left\lfloor \frac{1}{2}(a + b + c_{\text{in}}) \right\rfloor, \quad (4)$$

so that $a + b + c_{\text{in}} = s + 2k$ at every one of the eight states. That is the whole of the specification: add three bits and read the result in binary. For the partial product, on $(u, v) \in \{0, 1\}^2$, the relation is the product itself,

$$p = uv. \quad (5)$$

That s is an exclusive or and k a majority is a fact about these two maps, not information supplied with them.

The remaining two relations are single steps of a scan, each carrying a constant of the position rather than an extra input. For the range, one step of a running comparison against a bound bit β . Its state is the pair (ℓ, e) , meaning below the bound so far and equal to it so far, and with x the value bit at this position,

$$\ell_{\text{out}} = \ell \vee (e \wedge \bar{x} \wedge \beta), \quad e_{\text{out}} = e \wedge [x = \beta]. \quad (6)$$

In words, which is the level at which the source states it: once below the bound stay below, and while still equal let this position decide. For the exclusion, one step of a running inequality against a constant bit γ , whose state is the single bit d ,

$$d_{\text{out}} = d \vee [x \neq \gamma]. \quad (7)$$

Once a bit has differed the values differ, and this bit differs when it disagrees with the constant.

Four relations, then, stated as arithmetic and as nothing else. Each defines its f by its values and none by a construction, which is the distinction the experiment turns on. Because β and γ each take two values, the two scan relations are supplied twice over, so four relations become the six cells recovered below.

Each relation was then evaluated at all 2^n points of its input space — eight states for the adder and for the comparison step, four for the partial product and for the inequality step — and the graph so obtained was handed to index deconvolution. The method sees a table of outputs. It does not see a circuit, a gate name, or any indication that a particular one is expected. Whatever it returns is what the compiler is given, and there is no second route by which a hand-written gate could enter: the builders that assemble the multiplier, the comparator scan and the inequality scan read their gates from the recovery and contain no gate literals of their own.

It is worth seeing what that behaviour actually is, because it does not stay a table for long. Figure 3 follows the adder through the representation of Section 2. The enumerated relation is eight rows and two output columns, and that is the entire input (Panel A). Keeping only the positions at which an output is one turns each column into an index set (Panel B), and here the carry's set is $\{3, 5, 6, 7\}$: the same three-input majority index set that Section S3.1 obtains from voting, arrived at now from arithmetic. Dependence is then read off the set itself. A coordinate is essential when some single-bit flip crosses the set, and all three of the carry's do (Panel C); a coordinate is free when the set is closed under flipping it, which is visible as regularity. Panel D is the case where that decides the answer: against a zero bound bit the comparison cell's set is $\{1, 3, 5, 7\}$, which is the single schema $1**$ with anchor 1 and offsets $\{0, 2, 4, 6\}$, so no flip of e or x can leave it and the method returns the wire.

The same figure marks the limit of the representation. The sum bit's index set $\{1, 2, 4, 7\}$ admits no schema with a free position at all, so its cover is four single states and nothing is compressed. Parity is the standard example of that, and it is a useful reminder that the compression is a property of the behaviour rather than a service the method performs on request.

Six cells came back, and Table 2 sets each one against what was said to obtain it. These expressions, and no others, are what the answers below are built from.

Three features of that table are worth drawing out, because they are the reason the experiment was worth running rather than assumed.

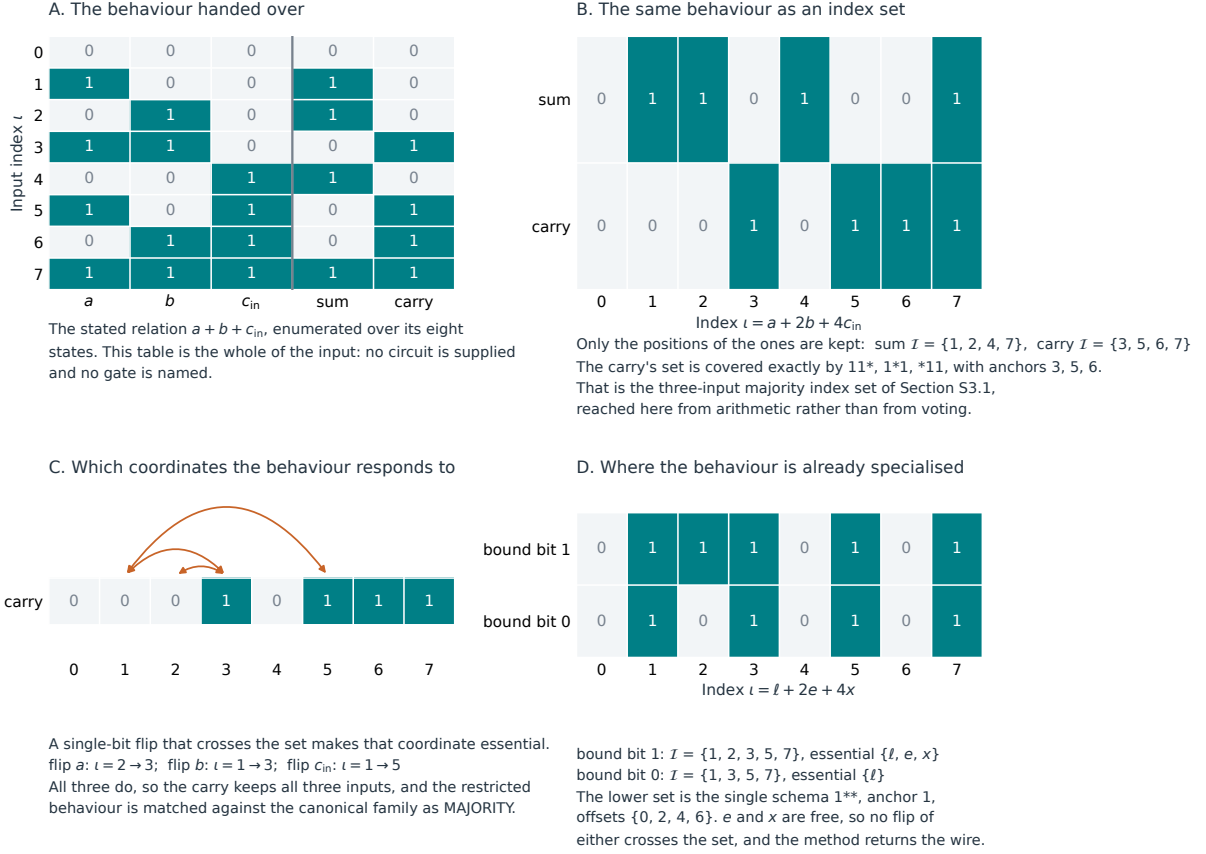


Figure 3: What is index deconvolution handed, and what does it read from it? Panel A is the complete input for the adder: the stated relation $a + b + c_{in}$ enumerated over its eight states, with no circuit and no gate name. Panel B keeps only where each output is one, which is the index set; the carry's is the three-input majority index set, covered exactly by the schemata 11*, 1*1, *11 with anchors 3, 5 and 6. Panel C reads dependence off that set: each orange arc is a single-bit flip that crosses it, so the coordinate flipped is essential, and all three are. Panel D is the comparison cell at each bound bit; against a zero bound bit the set collapses to one schema and two coordinates become free, which is the specialisation reported in the text. Every index set, cover and essential set in this figure is computed from the stated relations and checked against the recovered gates.

Table 2: The gates the method returned, against what was said to obtain them. The middle column names the relation, stated as Eqs. (4)–(7) and supplied as nothing but its graph; that is the entire input to the experiment. The right-hand column is the expression index deconvolution returned, on the inputs it found essential. Here ℓ and e are the incoming “less” and “equal” of a comparison, d the incoming “differs” of an inequality, and x the value bit at the current position; a bar denotes negation. The bound and constant bits are constants of the position, so each scan cell is recovered twice, once for each.

Cell	Stated as	Output	Expression returned
Full adder	$a + b + c_{in}$	sum	$a \oplus b \oplus c_{in}$
		carry	$\text{MAJ}_3(a, b, c_{in})$
Partial product	uv	product	$u \wedge v$
Comparison, bound bit 1	comparison scan	less	$\ell \vee e\bar{x}$
		equal	$e \wedge x$
Comparison, bound bit 0		less	ℓ
		equal	$e \wedge \bar{x}$
Inequality, constant bit 0	inequality scan	differs	$d \vee x$
Inequality, constant bit 1		differs	$d \vee \bar{x}$

The first is the adder. Handed the integer $a + b + c_{\text{in}}$ and nothing else, the method found all three inputs essential for both outputs and returned the sum as XOR and the carry as MAJORITY. That is the classical full adder, and neither name appears in what it was given. Figure 4 shows the whole of that recovery: the eight rows handed over, and the network handed back.

The second is that the comparison cell specialises itself. Against a bound bit of one it uses all three inputs. Against a bound bit of zero it reports that two of the three are inessential and returns the incoming “less” alone — below the bound one stays below, and no zero bound bit can create a new strict inequality. The “equal” output splits the same way, into $e \wedge x$ against a one and $e \wedge \bar{x}$ against a zero, which is the statement that equality survives exactly when the value bit agrees with the bound bit. Figure 5 shows the dependencies that vanish. None of this was applied afterwards as an optimisation; the cofactor test simply did not find those dependencies.

The third is a limit, and it is the same one that applies to Q2. What is recovered is a function; the name attached to it is a canonical choice inside an equivalence class of names. The sum bit is named uniquely, but the carry’s exact match class also contains KOFN and a two-clause regulatory form, and the unary “less” cell matches seven families at once, because a plain wire is expressible in almost all of them. The class is recorded for every cell in Supplementary Section S4. Nothing downstream depends on the name, since it is the function that is compiled.

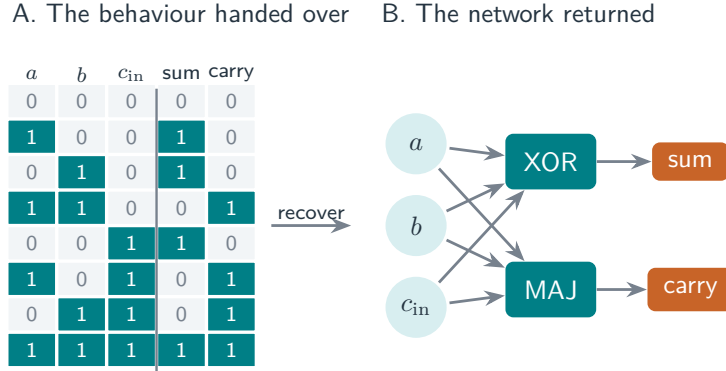


Figure 4: How is the full adder recovered? Panel A is everything the method is given: the complete behaviour of the stated relation $a + b + c_{\text{in}}$ over its eight input states. No circuit is supplied. Panel B is what comes back. Every one of the three inputs is found essential for both outputs, and the two gates are named against the canonical family: sum as XOR, carry as MAJORITY. The sum is named uniquely, while the carry’s exact match class also contains KOFN and a two-clause regulatory form, which are the same function under other names. These two gates, not any written by hand, are what Q7 compiles.

4.2 What the recovered gates force in the field

Two things are evident from what came back. First, every object the method returned is a gate over *bit* coordinates: the essential sets it reports are sets of bit positions, and the gates it names act on values that are zero or one. Second, the questions themselves are about *integers* — a 64-bit bound, a canonical field element, a product of two factors — so the recovered network and the question do not speak about the same objects until they are joined.

Joining them inside $\mathbb{F}_P$ leaves no freedom, and this is where the two algebraic components come from. A constraint variable ranges over the whole field, so a wire intended to carry one of the method’s bits must be pinned to $\{0, 1\}$. In a single quadratic row there is exactly one way to say that:

$$b(b - 1) = 0, \tag{8}$$

the unique monic quadratic whose roots are zero and one, and which has no other solutions because a field has no zero divisors. The compiler therefore emits Eq. (8) once for every input

A. Bound bit 1: every input matters

B. Bound bit 0: two dependencies vanish

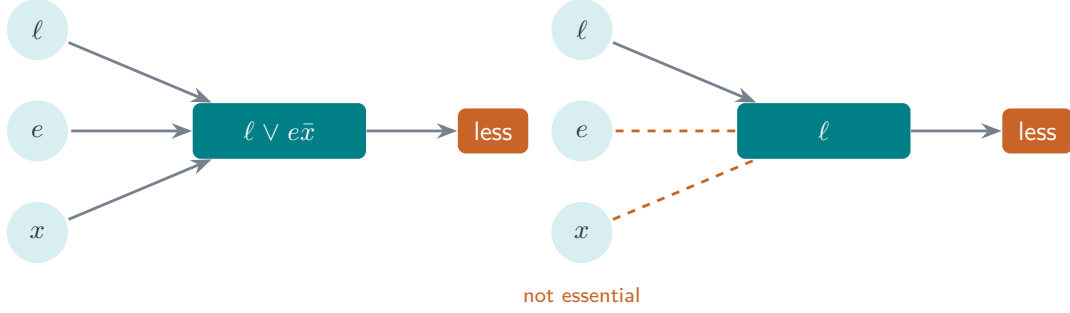


Figure 5: The comparator specialises itself. Both panels show the outgoing “less” of one comparison step, whose inputs are the incoming “less” ℓ , the incoming “equal” e , and the value bit x . The cell is stated only as the meaning of a running comparison; the bound bit is a constant of the position, and nothing else is said. Against a bound bit of one the method finds all three inputs essential and names a two-clause form. Against a bound bit of zero it finds e and x inessential and returns ℓ alone: below the bound one stays below, and no zero bound bit can create a new strict inequality. The orange dashed edges are the dependencies the method removed. They were not removed by an optimisation pass; the cofactor test simply did not find them.

bit of every system built this way. The integer the question asks about must then be tied to those same bits, which is the weighted sum

$$x = \sum_{i=0}^{w-1} 2^i b_i. \quad (9)$$

Neither equation was chosen for convenience; both are what the recovered network needs in order to exist as field constraints at all.

Why the sum is an integer and not a residue. Equation (9) is an identity in $\mathbb{F}_P$, so on its own it determines x only up to a multiple of the modulus, and a field circuit cannot distinguish x from an external $x + P$. It states what the question means only when the sum cannot wrap: with w bits the right-hand side lies in $[0, 2^w - 1]$, so if $2^w < P$ then that range sits inside $[0, P - 1]$ and the canonical representative of the sum equals x as an integer. This is why each width here is chosen against the modulus rather than against the question alone — 65 bits for Q5, 254 for Q6, 64 per factor for Q7 — and why the Q8 construction of Section 4.5 must split a 4096-bit integer into limbs instead: there no single field element can hold it, and the bound discipline replaces Eq. (9).

One consequence is worth drawing out. For Q5 I take the decomposition one bit wider than the bound, at 65 bits rather than 64. A 64-bit decomposition would make the range true by arithmetic, since no 64-bit sum can reach 2^{64} , and the recovered comparator would then be verifying something it could not fail. At 65 bits an out-of-range value is representable, and the comparator is what rejects it.

4.3 From a recovered gate to a quadratic row

Between the expression the method returns and the row the challenge evaluator reads there is one more step, and it is the step at which a hand-written gate could still slip in. A recovered gate names a function of whatever arity it found essential, while the constraint layer carries only two- and three-input gates, so the expansion between them is an obligation rather than a convenience. It is discharged by identity, not assumed: the expanded network is evaluated

back into a decision-diagram root, the named gate is built independently by the engine’s own gate builder, and the two roots are compared. Equal roots settle agreement on every input at once. All nine expansions in use are discharged this way, along with every gate the method can return at the arities in use and every function of one to three variables. Figure 6 shows one such expansion and the identity behind it. A wrong fold cannot reach a constraint system, because the two roots would differ and the check would fail.

Once the expansion is settled the lowering is mechanical. It carries AND, OR, XOR, NOT, three-input MAJORITY and the Boolean constants, and each becomes a fixed pattern of rows — $ab = z$ for a conjunction, $2ab = a + b - z$ for an exclusive or — with Eq. (8) emitted for every input, output and auxiliary signal. Repeated operands are combined as coefficients in the linear expressions, so fan-out and duplicate ports obey the same contract. Witness generation and constraint enforcement are kept apart throughout: the serialized rows, not the program that fills them in, are the acceptance boundary.

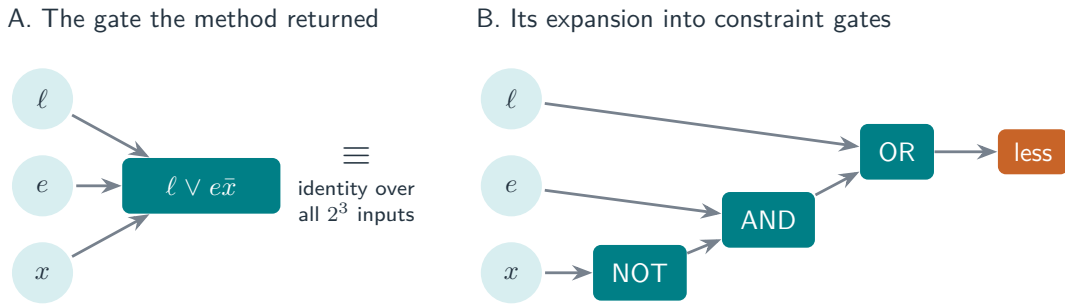


Figure 6: How does a recovered gate reach the constraint layer? Panel A is what deconvolution returned for one comparison step: a single gate of arity three, named as a two-clause form. The constraint layer does not carry that gate, so it must be folded into gates the layer does carry, shown in Panel B. The fold is not assumed correct. The expanded network is evaluated back into a decision-diagram root, the named gate is built independently by the engine’s own gate builder, and the two roots are compared: equal roots settle agreement on all eight inputs at once. A wrong fold cannot reach a constraint system, because the two roots would differ and the check would fail.

4.4 The three compiled systems, and what each one verifies

Three of the four questions are answered by scanning the recovered cells across the width the question asks for. Q5 threads the comparison cell through a 65-bit decomposition and reads off the range predicate $x < 2^{64}$: 335 rows over 200 signals. Q6 runs the same comparison cell at 254 bits, against P itself so that the element is canonical, together with the recovered inequality cell against the constant one: 2,193 rows over 1,223 signals. Q7 is a shift-and-add multiplier assembled from the recovered full adder and partial product at the full 64 bits, one adder per column with every product bit and every stage carry retained: 107,271 rows over 53,572 signals.

Each was then checked against an independent integer predicate, exhaustively wherever exhaustion is possible and adversarially where it is not. For Q5 the scan was rebuilt at every width up to eight and run against every bound and every input — $\sum_{w=1}^8 4^w = 87,380$ cases, all correct. For Q7 the multiplier was enumerated at widths one to five, where the full $2w$ -bit product and the zero stage carries must both come out right, and the compiled rows themselves were then swept at widths four, five and six over every triple (u, v, n) : $4,096 + 32,768 + 262,144 = 299,008$ triples, of which 216 satisfy the relation, with no discrepancy against the predicate in either direction.

That ladder also gives an exact law, and the law is what carries the evidence up to 64 bits. A width- w multiplier compiles to $26w^2 + 12w + 7$ rows and $7w^2 + 1$ gates, fitted over widths four to sixty-four and confirmed at 128. At $w = 64$ the row law reads 107,271, which is exactly the system built above. At 64 bits enumeration is out of reach — 2^{192} triples — so the system is

attacked instead: it accepts genuine factorisations, including semiprimes whose product occupies all 64 bits, and rejects trivial factors, wrong products, and a pair of 34-bit factors whose product overflows 64 bits and would otherwise pass as its truncated residue.

Two shorter relations, kept as checks. The field can express two of these conditions directly and far more cheaply. For $r \neq 1$ one may impose $(r - 1)s = 1$, which has a witness exactly when $r - 1$ is invertible, and at $r = 1$ would read $0 = 1$; for Q7 one may bound u, v to 64 bits and impose $uv = n$, which is an integer equality as well as a modular one because $uv < 2^{128} < P$. I keep both, not as the answer, but because a one-row relation that borrows the field's own arithmetic is a useful independent check on a hundred-thousand-row system that borrows none of it. Neither route finds factors; both verify factors that are supplied.

The recovered route is larger than the direct one at every question — 335 rows against 65 at Q5, 2,193 against one at Q6, and at Q7, where the comparison is like for like at width 64, 107,271 against 200. No efficiency claim is made for it, and none is intended. What it buys is that the gates performing the arithmetic were obtained from behaviour rather than asserted, and Figure 7 is the whole of that chain.



Figure 7: From a stated relation to verified constraints. The cell enters as an integer relation, not as a circuit. Deconvolution names its gates, the expansion into the constraint layer's gate set is verified by identity, and the resulting rows are replayed against an independent predicate. No gate in this chain is written down by hand.

4.5 A large factorisation and the role of carries

A 4096-bit integer exceeds the field modulus, so one field element cannot hold it. For Q8, split each integer into 64 pieces, or *limbs*, in base $B = 2^{64}$. Each limb is a 64-bit integer:

$$u = \sum_{i=0}^{63} u_i B^i, \quad v = \sum_{i=0}^{63} v_i B^i, \quad n = \sum_{i=0}^{63} n_i B^i. \quad (10)$$

Multiply the limbs and collect terms by position, exactly as in written multiplication. For each column k ,

$$\sum_{i+j=k} u_i v_j + c_k = n_k + B c_{k+1}. \quad (11)$$

The incoming carry is c_k ; the outgoing carry is c_{k+1} . Set the initial and final carries to zero, and set output limbs above position 63 to zero. Figure 8 illustrates how the column equations fit together, using decimal digits so every partial product and carry can be followed by hand. The construction above uses the same pattern in base 2^{64} .

The bounds are part of the argument. Intermediate carries are restricted to 70 bits. Both sides of Eq. (11) are then below $2^{135} < P$, so the field cannot hide a wraparound in any column. Conversely, ordinary multiplication produces carries within these bounds. Together with nontrivial factors and the zero upper outputs, this proves the equivalence between a satisfying assignment and the intended integer factorisation.

Checks evaluate these equations directly, including altered bits, partial products, carries and public values. A useful negative control removes the carry bounds: the false integer statement $2((P + 7)/2) = 7$ can then satisfy the multiplication columns modulo P . Restoring the bounds

Follow the partial products and carries: $23 \times 47 = 1081$ (base 10)

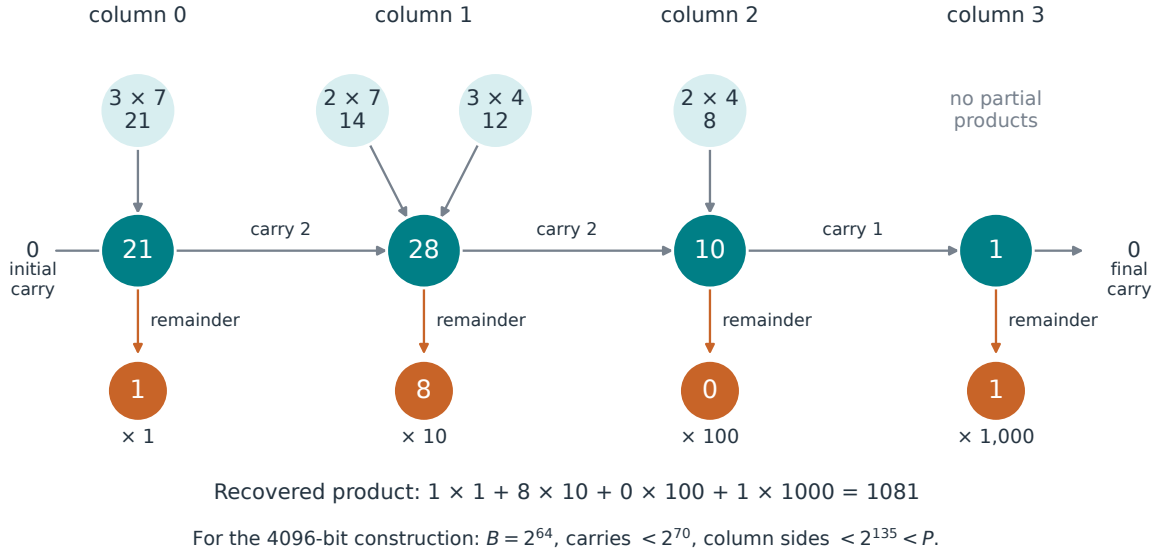


Figure 8: How do partial products become an exact integer product? The decimal example $23 \times 47 = 1081$ makes the carries visible. Each pale node is a partial product. Teal nodes add the products in one column and its incoming carry. Division by 10 sends the quotient to the next column and keeps the orange remainder as an output digit. Columns run from least to most significant, so the digits read 1, 8, 0, 1 in that order. The large construction uses the same equations in base 2^{64} ; its limb and carry bounds keep each column equality below the field modulus.

rejects it. Supplementary Section S6 gives the completeness and soundness proofs, and Section S7 the compiled verification record.

4.6 Where the recovered route stops, and why

Q8 is answered above, and the recovered route of the previous subsection is not what answers it. It is worth being precise about why, because the reason is not effort.

The obvious attempt is to build a 4096-bit multiplier from the same recovered cells that give Q7. Its cost follows an exact law, fitted over widths four to sixty-four and confirmed at 128: a width- w multiplier has $7w^2 + 1$ gates and $26w^2 + 12w + 7$ rows. At $w = 4096$ that is 436,256,775 rows against 25,725 for the limb construction, a factor of about seventeen thousand. Large, but on its own only an argument about expense.

The real obstruction appears when we ask what the recovery has to hold in memory rather than what the circuit costs. Deconvolution's reach comes from the decision diagram staying small, and whether it does is a property of the function, not of the implementation. Table 3 puts the two cases side by side on the same engine, and compares like with like: in both columns the figure is the allocation, every node the recovery had to hold, which is the quantity that decides whether the run finishes at all. Majority's allocation grows polynomially, as $n^{3.96}$ across six sizes, which is what carries it to 151 inputs; the recovered function's own description is far smaller still, 5,776 nodes at $n = 151$ and growing as $n^{1.97}$. Multiplication's allocation grows by a factor of 2.977 for every additional bit, measured across eleven widths, and is already beyond a budget of eight million nodes at thirteen. Extrapolating that factor to 4096 bits gives some 10^{1942} nodes. The observable universe holds about 10^{80} atoms.

This is not a gap awaiting a better encoding. The exponential growth of such descriptions for integer multiplication is a known lower bound [11], and the measurements above are that bound

Table 3: Why one function is recoverable at scale and the other is not. Both columns are the same engine on the same machine, and both report allocation: every decision node the recovery had to hold. A majority need only remember how many inputs are active, so the growth stays polynomial and the recovered function’s own description is smaller again, $(n + 1)^2/4$ nodes. A multiplier must retain every partial sum, and no summary suffices; the growth is geometric and the limit is intrinsic.

	Majority	Multiplication
Allocation growth	polynomial, $n^{3.96}$	geometric, $\times 2.977$ per bit
Measured over	6 sizes, $n = 31$ to 151	11 widths, $w = 2$ to 12
Largest built	41,196,123 nodes	4,384,542 nodes
At that size	$n = 151$, all 2^{151} states	$w = 12$ bits
Recovered description	5,776 nodes, $n^{1.97}$	not reached
Extrapolated	—	$\approx 10^{1942}$ nodes at 4096 bits

appearing in our own engine. The limb construction avoids it entirely by never going through Boolean logic: each limb product is one field multiplication, and the blow-up has no opportunity to occur. Reporting the boundary is more useful than omitting the question, since it says exactly which problems this method is the right instrument for.

5 What this response establishes

The solutions give a majority construction for every odd input count and quadratic systems equivalent to the requested integer conditions. Their assumptions determine what each answer means.

What holds these together is that one method supplies them. Index deconvolution recovers functional dependence from behaviour and, run on a canonical decision diagram, does so without writing the repertoire down: majority comes back exactly at 151 inputs, and the arithmetic cells that build Q5, Q6 and Q7 come back from stated integer relations with no gate named by hand. The arithmetic construction then preserves integer meaning inside a modular system by bounding every place where wraparound could occur.

The same account marks where the method does not reach. Q8 is answered by the limb construction and not by recovery, and the reason is measured rather than conceded: descriptions of integer multiplication grow geometrically in the width, so at 4096 bits no such description can be written down. A method worth using is one whose boundary is known, and this response reports both sides of it.

The finite evidence supports these specific constructions. It includes small majority circuits, the recovered ladder to 151 inputs and checks of compiled arithmetic constraints. A complete 2025-input majority circuit and generated zero-knowledge proofs remain outside that evidence. Supplementary Information separates the general proofs from these finite results and provides the details needed to reproduce and assess them.

References

- [1] 0xPARC. *Beyond Primitive Computing: Puzzles and Connections*, Field Extension. <https://fieldextension.org/3b1b/>.
- [2] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, *Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying*, arXiv:1904.10393 [q-bio.NC] (2019). <https://arxiv.org/abs/1904.10393>.

- [3] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, “Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying,” in *Handbook of Unconventional Computing*, Vol. 1: Theory, ed. A. Adamatzky (World Scientific, 2021), Chap. 5, pp. 171–220. https://doi.org/10.1142/9789811235726_0005.
- [4] J. H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence* (University of Michigan Press, Ann Arbor, MI, 1975).
- [5] A. N. Kolmogorov, *Three approaches to the definition of the concept of quantity of information*, *Problems of Information Transmission* 1(1), 3–11 (1965); English reprint, *Three Approaches to the Quantitative Definition of Information*, *International Journal of Computer Mathematics* 2(1–4), 157–168 (1968).
<https://doi.org/10.1080/00207166808803030>.
- [6] G. J. Chaitin, *On the length of programs for computing finite binary sequences*, *Journal of the ACM* 13(4), 547–569 (1966).
<https://doi.org/10.1145/321356.321363>.
- [7] M. Li and P. M. B. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed., Texts in Computer Science (Springer, Cham, 2019).
<https://doi.org/10.1007/978-3-030-11298-1>.
- [8] G. Cohen, I. B. Damgård, Y. Ishai, J. Kölker, P. B. Miltersen, R. Raz, and R. D. Rothblum. *Efficient Multiparty Protocols via Log-Depth Threshold Formulae*, 2013. https://www.wisdom.weizmann.ac.il/~ranraz/publications/Pmajority_mpc-1.pdf.
- [9] J. H. Cheon, K. Han, and M. Hhan. *Faster Homomorphic Discrete Fourier Transforms and Improved FHE Bootstrapping*. Cryptology ePrint 2018/1073. <https://eprint.iacr.org/2018/1073>.
- [10] A. Hernández-Espinosa, *CausalBool*, source code accompanying this response, 2026.
- [11] R. E. Bryant. *On the Complexity of VLSI Implementations and Graph Representations of Boolean Functions with Application to Integer Multiplication*. IEEE Transactions on Computers 40(2):205–213, 1991.
- [12] iden3. *Circom 2.2.3* and *snarkjs 0.7.6*. <https://docs.circom.io/>; <https://www.npmjs.com/package/snarkjs/v/0.7.6>.